

UNIVERSIDAD PRIVADA DOMINGO SAVIO
FACULTAD DE INGENIERÍA
CARRERA DE INGENIERÍA EN SISTEMAS
ASIGNATURA: INGENIERÍA DE SOFTWARE I

ACTIVIDAD 03 — ÁLBUM DE MODELOS UML

CRITERIO DE VERIFICACIÓN #2 (30 PUNTOS) — GATE 3 DEL SDLC

"YAPU": DESCRIPCIÓN DEL DISEÑO DE SOFTWARE (IEEE STD 1016-2009 SDD)

**MAPA INTEGRAL DE ARQUITECTURA, CLASES ESTRUCTURALES,
PERSISTENCIA MULTIBASE DE DATOS (INDEXEDDB Y CLOUD FIRESTORE),
CASOS DE USO Y COMPORTAMIENTO**

NOMBRE DE LOS ESTUDIANTES

Jhoel Alvaro Cruz Zurita

Emmanuel Ponce Quiroga

DOCENTE

Ing. Requena Llorentty Jimmy Nataniel

Santa Cruz de la Sierra - Bolivia

Septiembre de 2026

RESUMEN EJECUTIVO

El presente informe formal documenta la Actividad 03: Álbum de Modelos UML correspondiente al Criterio de Verificación #2 (30 puntos) de la asignatura Ingeniería de Software I en la Universidad Privada Domingo Savio (Sede Santa Cruz de la Sierra). Enmarcado en el Gate 3 del ciclo de vida de desarrollo de software (SDLC), el documento adopta el estándar internacional IEEE Std 1016-2009 (Software Design Description - SDD) para formalizar la arquitectura y el diseño técnico del proyecto YAPU (Sembradío en lengua quechua), una Plataforma Web Progresiva (PWA) de alta resiliencia concebida para la enseñanza y revitalización de la lengua originaria quechua (variante sureña Collao/Chuquisaca) en el Estado Plurinacional de Bolivia.

A lo largo del documento se operacionalizan los 10 Requisitos Funcionales (RF-001 al RF-010), los Requisitos No Funcionales (RNF-001 al RNF-005) y los Requisitos de Sostenibilidad (RS-001 al RS-004) formalizados previamente en la Especificación de Requisitos de Software (IEEE Std 830-1998). La solución se estructura en cuatro dimensiones arquitectónicas: (1) Vista Arquitectónica de 4 Capas con comunicación HTTP/2 y TLS 1.3 mediada por Service Worker (sin rombos de decisión condicional en la red); (2) Vista Estructural de Clases del Dominio con Inversión de Dependencias (SOLID) y composición estricta; (3) Vista Estructural de Persistencia Multibase de Datos que resuelve mediante el patrón Data Mapper la coexistencia entre la base de datos local en el navegador (IndexedDB con ObjectStores e índices) y la base de datos documental remota (Cloud Firestore NoSQL con subcolecciones); (4) Vista Funcional y Dinámica con casos de uso (include/extend), diagramas de secuencia en 5 fases, autómata de estados y algoritmos deterministas en TypeScript que aseguran coste cero (\$0) en APIs, latencia menor a 50 ms y cero alucinaciones lingüísticas.

Asimismo, en cumplimiento estricto de las exigencias éticas y de transparencia de la UPDS, se incluye una exhaustiva Matriz de Auditoría de Inteligencia Artificial que transparenta el uso de herramientas de IA como asistentes de diseño, documentando las decisiones aceptadas, las modificaciones críticas y los rechazos expresos efectuados por los estudiantes, acompañados de la evidencia de refinamiento de prompts y el análisis crítico de alucinaciones.

CAPÍTULO I: INTRODUCCIÓN Y MARCO NORMATIVO

1.1 El Estándar IEEE Std 1016-2009 (Software Design Description)

El estándar IEEE Std 1016-2009 define las directrices y puntos de vista recomendados para la descripción del diseño de software. Un SDD traduce los requerimientos del sistema en especificaciones de arquitectura, estructuras de datos, interfaces y algoritmos que sirven como puente riguroso entre la fase de especificación (Gate 2) y la fase de construcción (Gate 4).

1.2 Metodología de las 5 Secciones por Diagrama

Para garantizar un análisis de ingeniería exhaustivo y uniforme, cada diagrama de este álbum desarrolla cinco secciones técnicas obligatorias:

1. Resumen Ejecutivo: Propósito arquitectónico y alcance específico del modelo.
2. Diccionario de Clases / Elementos / Schemas: Definición de atributos con justificación técnica de tipos de datos y signaturas de operaciones.
3. Análisis de Relaciones: Justificación técnica formal de por qué se empleó composición, agregación, herencia, realización o dependencia aplicando Principios SOLID.
4. Seguridad, Integridad y Gobernanza: Políticas de protección de datos, aislamiento de red, inmutabilidad curricular y reglas de acceso.
5. Guía de Explicación para Evaluación Oral: Argumentos clave y respuestas técnicas frente a preguntas trampa del tribunal docente.

1.3 Contexto del Problema y Delimitación de YAPU

YAPU aborda el fenómeno sociolingüístico de la pérdida de hablantes de quechua en Chuquisaca. Para garantizar inclusión en áreas rurales y periurbanas, el sistema impone restricciones técnicas críticas: operar en smartphones de gama de entrada con Android 8.0 con menos de 150 MB de memoria RAM en ejecución (RS-002), funcionar plenamente sin internet (RF-009) y prescindir de LLMs generativos externos para evitar costos y alucinaciones lingüísticas.

CAPÍTULO II: DIAGRAMA 1 — ARQUITECTURA GENERAL DEL SISTEMA

2.1 Representación Arquitectónica (Mermaid)

```
graph TB
    subgraph CLIENT_TIER [1. Capa de Cliente: PWA Ligera < 150MB RAM]
        UI[Interfaz Astro SSG + Islands Architecture]
        SW[Service Worker & Workbox Fetch Interceptor]
        CACHE_API[(Cache Storage API: App Shell & WebP)]
        IDB_LOCAL[(IndexedDB Local: Evaluaciones y Cola)]
        subgraph CORE_AI [Motor IA Determinista en TypeScript]
            GEN[Generador de Exámenes por Permutación]
            DIST[Filtro de Distractores Semánticos]
            EVAL[Evaluador Local de Aciertos]
        end
    end

    subgraph CLOUD_TIER [2. Servicios Gestionados: Google Firebase]
        FAUTH[Firebase Auth JWT & Roles RBAC]
        FSTORE[Cloud Firestore NoSQL Engine]
        RULES[firestore.rules: Seguridad Declarativa]
    end

    subgraph VPS_TIER [3. Infraestructura Autogestionada: VPS UPDS]
        NGINX[Reverse Proxy Nginx HTTP/2]
        ASSETS[Repositorio Assets WebP < 100KB]
        OPEN_DATA[Exportador Corpus Lingüístico CSV/JSON]
    end

    UI --> SW
    UI --> CORE_AI
    SW --> CACHE_API
    SW --> IDB_LOCAL
    SW -->|HTTPS TLS 1.3 REST| FAUTH
    SW -->|HTTPS TLS 1.3 / gRPC| FSTORE
    SW -->|HTTPS TLS 1.3 HTTP/2| NGINX
    IDB_LOCAL -. ->|Background Sync API| FSTORE
```

2.2 Desglose Técnico IEEE 1016

1. Resumen Ejecutivo

Modela la topología física, lógica y de comunicaciones de YAPU, estructurada en tres niveles físicos y cuatro capas lógicas. El diseño elimina intermediaciones innecesarias: la aplicación emite peticiones HTTP estándar que son interceptadas de manera desacoplada por el Service Worker, garantizando que el software permanezca 100% operativo sin internet.

2. Diccionario de Componentes y Nodos

Componente	Capa / Tecnología	Justificación Técnica de Ingeniería
Astro SSG + Islands	Cliente PWA	Renderizado estático en build time; solo hidrata islas interactivas activas. RAM < 150MB y FCP < 1.8s.
Service Worker & Workbox	Cliente Web Worker	Intercepta el tráfico HTTP; sirve App Shell desde Cache Storage y canaliza transacciones a IndexedDB.
Motor IA en TypeScript	Cliente Local	Ejecutado enteramente en el navegador del teléfono móvil sin latencia de red ni consumo de cuotas.
Cloud Firestore NoSQL	Cloud Firebase BaaS	Base documental distribuida multirregión con absorción de hasta 500 sesiones concurrentes (RNF-005).
Reverse Proxy Nginx	VPS Linux UPDS	Servidor HTTP/2 sobre Debian 12 con compresión Brotli/Gzip para imágenes WebP < 100KB y datos abiertos.

3. Análisis de Relaciones e Interfaces de Comunicación

- Conectores HTTP/2 y TLS 1.3: Todo el tráfico entre el Service Worker y los servicios remotos (Firebase y Nginx) viaja cifrado con suite AES-GCM-256 sobre HTTPS.
- Background Sync API: Conector asíncrono no bloqueante entre IndexedDB y Firestore. Si la conexión cae, las evaluaciones quedan almacenadas en la memoria local y el navegador dispara la sincronización automáticamente al detectar conectividad.

4. Seguridad y Gobernanza

- Reglas Declarativas (`firestore.rules`): Validan en el servidor que las escrituras coincidan con el UID del token JWT.
- Aislamiento de Datos Abiertos (RS-004): El endpoint del VPS exporta el corpus léxico en CSV/JSON previa purga de cualquier dato de identificación personal.

5. Guía de Explicación para Evaluación Oral

- Pregunta: ¿Por qué en la arquitectura no hay un rombo de decisión para comprobar si hay internet antes de conectarse?

Respuesta: Porque un diagrama de arquitectura describe componentes, límites y protocolos de red, no control de flujo procedimental. En una PWA, la UI siempre despacha peticiones estándar HTTP; es el Service Worker quien resuelve si responde desde la Cache API o delega a la red. Colocar un rombo de decisión en la red es un error conceptual que confunde arquitectura con un diagrama de actividades.

CAPÍTULO III: DIAGRAMA 2 — CLASES DEL DOMINIO

PEDAGÓGICO Y MOTOR IA

3.1 Representación de Clases Orientada a Objetos (Mermaid)

```
classDiagram
    namespace PresentationAndState {
        class EvaluationController {
            -IEvaluationEngine evaluationEngine
            -IOfflineFirstRepository repository
            -StudentSession activeSession
            +initExam(levelId: int) void
            +submitAnswer(questionId: String, selectedOption: String) void
            +finalizeExam() EvaluationResult
        }
        class StudentSession {
            -String studentId
            -int currentLevel
            -int currentStreak
            +updateStreak() void
        }
    }
    namespace CoreLearningDomain {
        class Evaluation {
            -String evaluationId
            -String studentId
            -int levelId
            -int scorePercentage
            -boolean isSynchronized
            +calculateScore() int
            +isPassed() boolean
        }
        class Question {
            -String questionId
            -String statementText
            -String correctOption
            -List~String~ distractors
            -boolean isCorrect
            +evaluateAnswer(selected: String) boolean
        }
    }
    namespace SecurityAndDeterministicAI {
        class IEvaluationEngine {
            <<interface>>
            +generateAssessment(levelId: int, baseCorpus: List~BaseSentence~)
            List~Question~
            +validateAnswerDeterministically(qId: String, selected: String) boolean
        }
        class DeterministicAIEngine {
```

```

    -int PASSING_THRESHOLD = 70
    -int ITEMS_PER_TEST = 10
    +generateAssessment(levelId: int, baseCorpus: List~BaseSentence~)
List~Question~
}
class BaseSentence {
  -String authorTeacherId
  -String validatorTeacherId
  -String culturalContext
  +isApprovedForExams() boolean
}
}
EvaluationController ..> IEvaluationEngine : DIP
IEvaluationEngine <|.. DeterministicAIEngine : Realización
Evaluation "1" *-- "10" Question : Composición Estricta
EvaluationController o-- "1" StudentSession : Agregación

```

3.2 Desglose Técnico IEEE 1016

1. Resumen Ejecutivo

Modela la estructura estática del dominio pedagógico de YAPU, encapsulando la lógica de evaluación y generación algorítmica mediante abstracciones orientadas a objetos y bajo acoplamiento.

2. Diccionario de Clases y Justificación de Tipos

Clase / Interfaz	Atributos Críticos y Tipos	Justificación Técnica de Tipos y Operaciones
EvaluationController	IEvaluationEngine engine StudentSession session	Controlador GRASP desacoplado. Depende de interfaces para permitir pruebas unitarias automatizadas con Mocks.
Evaluation	String evaluationId int scorePercentage	evaluationId tipado como String UUIDv4. scorePercentage se maneja como int [0..100] para evitar errores de precisión flotante.

Question	List<String> distractors boolean isCorrect	Modela cada reactivo del examen. Contiene exactamente 3 distractores verosímiles de la misma categoría gramatical.
IEvaluationEngine	generateAssessment() validateAnswer()	Contrato abstracto de evaluación. Permite sustituir la implementación del motor sin alterar la UI ni la controladora.

3. Análisis de Relaciones SOLID

- Composición Estricta (`Evaluation \*-- Question`): Las 10 preguntas carecen de ciclo de vida o identidad ontológica fuera de la evaluación en que se generaron; si la evaluación se destruye o reinicia, las preguntas se purgan simultáneamente.
- Agregación (`EvaluationController o-- StudentSession`): La sesión del estudiante preexiste a cualquier examen individual y sobrevive a él.
- Inversión de Dependencias (DIP): `EvaluationController` depende de `IEvaluationEngine`, no de la clase concreta `DeterministicAIEngine`.

4. Seguridad y Determinismo Algorítmico

El motor opera con funciones puras en TypeScript. A partir de una semilla determinista y un banco estático certificado, la salida es matemáticamente predecible y reproducible, eliminando por completo cualquier riesgo de alucinación lingüística.

5. Guía de Evaluación Oral

- Pregunta: ¿Por qué la relación entre Evaluation y Question no es agregación?

Respuesta: Porque una agregación implicaría que una pregunta puede existir de forma independiente sin una evaluación. En YAPU, las preguntas se sintetizan dinámicamente con permutaciones específicas para ese intento. No existe una pregunta huérfana en el sistema.

CAPÍTULO IV: DIAGRAMA 3 — PERSISTENCIA MULTIBASE DE DATOS (INDEXEDDB VS FIRESTORE)

4.1 Representación de Clases de Persistencia Multibase (Mermaid)

```
classDiagram
    namespace Local_IndexedDB_Schemas {
        class EvaluationStoreRecord {
            +String evaluationId : keyPath
            +int levelId : index_by_level
            +String syncStatus : index_by_status
            +int scorePercentage
            +String questionsPayloadJSON
        }
        class SyncQueueRecord {
            +String queueId : keyPath
            +String entityType : index_by_entity
            +String operationType
            +String payloadJSON
            +int retryCount
        }
    }
    namespace Remote_Firestore_Schemas {
        class FirestoreEvaluationDocument {
            +String docId : path_evaluaciones_id
            +String studentUid : indexed_field
            +Timestamp createdAt : server_timestamp
            +int scorePercentage
        }
        class FirestoreQuestionSubdocument {
            +String questionId : path_preguntas_id
            +String statementText
            +String correctOption
            +boolean isCorrect
        }
    }
    namespace MappersAndCoordination {
        class EvaluationLocalMapper {
            +toLocalRecord(domain: Evaluation) EvaluationStoreRecord
            +toDomain(record: EvaluationStoreRecord) Evaluation
        }
        class EvaluationRemoteMapper {
            +toFirestoreDocument(domain: Evaluation) FirestoreEvaluationDocument
            +toDomain(doc: FirestoreEvaluationDocument) Evaluation
        }
        class OfflineFirstEvaluationRepository {
            +saveEvaluation(eval: Evaluation) Promise~boolean~
            +synchronizePendingQueue() Promise~int~
        }
    }
```

```

    }
    OfflineFirstEvaluationRepository --> EvaluationLocalMapper
    OfflineFirstEvaluationRepository --> EvaluationRemoteMapper
    FirestoreEvaluationDocument "1" *-- "10" FirestoreQuestionSubdocument
  
```

4.2 Desglose Técnico IEEE 1016

1. Resumen Ejecutivo

Modela la arquitectura estructural que resuelve la convivencia entre dos motores de base de datos con paradigmas distintos: IndexedDB en el cliente móvil y Cloud Firestore NoSQL en la nube. Aplica los patrones Data Mapper y Offline-First Repository para mantener las entidades del dominio completamente independientes de la tecnología física de persistencia.

2. Diccionario de Schemas y Mappers

Elemento / Schema	Motor / Paradigma	Justificación Técnica de Ingeniería
EvaluationStoreRecord	IndexedDB (Cliente)	ObjectStore indexado. Usa evaluationId como keyPath e índices B-Tree. Serializa preguntas en JSON para I/O atómico en memoria flash.
SyncQueueRecord	IndexedDB (Cliente)	Cola persistente de mutaciones no sincronizadas. Registra reintentos y orden FIFO para el Service Worker.
FirestoreEvaluationDocument	Firestore (Nube)	Documento NoSQL en /evaluaciones/{id}. Almacena studentUid y Timestamp nativo de Google (server_timestamp) para auditoría.
EvaluationLocalMapper / RemoteMapper	Data Mappers	Clases puras de transformación bidireccional entre la entidad rica del

dominio y los registros físicos
planos de cada base de datos.

3. Análisis de Relaciones

- Desacoplamiento por Mappers: Las entidades de negocio no importan SDKs de Firestore ni librerías de IndexedDB. Toda traducción se aísla en los mappers.
- Composición NoSQL (`FirestoreEvaluationDocument *-- FirestoreQuestionSubdocument`): Representa una subcolección en Firestore. Permite a los docentes auditar reactivos específicos sin incurrir en lecturas masivas de documentos completos.

4. Resolución de Conflictos e Integridad

El repositorio unificado implementa la regla de Última Escritura Gana (Last-Write-Wins), comparando las marcas temporales locales con el `server_timestamp` emitido por Firestore para resolver colisiones al restablecer la conexión.

5. Guía de Evaluación Oral

- Pregunta: ¿Cómo se manejan los diagramas de clases estructurales cuando el sistema tiene múltiples bases de datos?

Respuesta: Se desacoplan en tres niveles: (1) Entidades de Negocio puras; (2) Schemas Físicos específicos para cada base de datos (ObjectStores en IndexedDB y Colecciones/Subcolecciones en Firestore); y (3) Data Mappers que convierten entre el dominio y cada schema, coordinados por un Repositorio Unificado que oculta la complejidad multibase al resto del sistema.

CAPÍTULO V: DIAGRAMA 4 — MODELO ENTIDAD-RELACIÓN (ERD) Y TAXONOMÍA

5.1 Representación del Diagrama Entidad-Relación (Mermaid)

```
erDiagram
    USUARIO ||--o{ PERFIL_ESTUDIANTE : "posee_perfil"
    USUARIO ||--o{ ORACION_BASE : "crea_como_autor"
    USUARIO ||--o{ ORACION_BASE : "certifica_como_validador"
    PERFIL_ESTUDIANTE ||--o{ PROGRESO_NIVEL : "registra_avance"
    NIVEL ||--o{ PROGRESO_NIVEL : "determina_nivel"
    NIVEL ||--o{ PALABRA_VOCABULARIO : "contiene_lexico"
    PERFIL_ESTUDIANTE ||--o{ VOCABULARIO_ESTUDIANTE : "gestiona_estudio"
    PALABRA_VOCABULARIO ||--o{ VOCABULARIO_ESTUDIANTE : "es_clasificada"
    NIVEL ||--o{ ORACION_BASE : "asocia_nivel"
    PERFIL_ESTUDIANTE ||--o{ EVALUACION : "rinde_evaluacion"
    NIVEL ||--o{ EVALUACION : "evalua_nivel"
    EVALUACION ||--|{ DETALLE_PREGUNTA : "genera_preguntas_composicion"
    ORACION_BASE ||--o{ DETALLE_PREGUNTA : "sirve_de_semilla"
    PERFIL_ESTUDIANTE ||--o{ RETO_COMUNITARIO : "propone_reto"
    USUARIO ||--o{ RETO_COMUNITARIO : "modera_reto"
```

5.2 Taxonomía de Relaciones de Persistencia

1. Relaciones Fuertes (Existencia Independiente): USUARIO a ORACION_BASE (doble asociación de autor y validador) y NIVEL a PALABRA_VOCABULARIO (catálogo léxico A1). Si un docente se retira, las oraciones aprobadas permanecen intactas en el banco histórico.
2. Relaciones Débiles por Existencia: USUARIO a PERFIL_ESTUDIANTE (1:0..1). Si un estudiante ejerce su derecho legal de supresión de datos (Habeas Data boliviano), su perfil académico se elimina en cascada.
3. Relaciones Débiles por Identificación: EVALUACION a DETALLE_PREGUNTA (composición pura 1:10) y PROGRESO_NIVEL (tabla asociativa N:M con clave primaria compuesta única id_estudiante + id_nivel).

CAPÍTULO VI: VISTA FUNCIONAL Y CASOS DE USO

6.1 Diagrama General de Casos de Uso (Diagrama 5)

Mapea los 10 Requisitos Funcionales del SRS frente a los 3 actores primarios: Estudiante, Docente Quechua y Administrador UPDS.

- Relaciones «include» Obligatorias: Registrar Cuenta incluye Verificar Token Correo; Modo Offline incluye Sincronizar Progreso al Reconectar; Gestionar Oraciones y Proponer Retos incluyen Doble Validación Docente.
- Relación «extend» Condicional: Rendir Evaluación extiende a Desbloquear Siguiente Nivel si y solo si la nota obtenida cumple la condición de guarda obligatoria [calificación $\geq 70\%$].

6.2 Motor de IA Determinista vs. LLMs Comerciales (Diagrama 6.1 y 9.1)

El motor determinista local en TypeScript ejecuta un autómata que descompone oraciones base en componentes sintácticos (Sujeto, Objeto, Verbo), aplica sufijos aglutinantes quechuas controlados (-pi, -man, -manta) y extrae 3 distractores del mismo campo semántico mediante Fisher-Yates Shuffle. Esto garantiza coste cero (\$0 en APIs), ejecución offline en smartphones rurales y cero alucinaciones gramaticales.

6.3 Protocolo de Doble Moderación y Principio de Cuatro Ojos (Diagrama 6.2)

Aplica la regla inviolable `autor_id != validador_id`. Ningún docente puede certificar sus propios reactivos pedagógicos. Todo aporte queda en estado 'pendiente' hasta que un segundo docente certificado valida la ortografía Collao y el contexto cultural etnográfico.

CAPÍTULO VII: VISTA DINÁMICA Y DE COMPORTAMIENTO

7.1 Diagrama General de Secuencia Temporal (Diagrama 7)

Modela el ordenamiento temporal a lo largo de 5 fases: (1) Carga inicial de App Shell desde Cache Storage; (2) Práctica léxica interactiva; (3) Generación y corrección de examen en local (< 50 ms); (4) Persistencia inmediata en IndexedDB; y (5) Sincronización asíncrona en segundo plano (Background Sync) con vaciado de lotes (Batch Write) hacia Cloud Firestore al recuperar conectividad.

7.2 Máquina de Estados Finitos (Diagrama 8)

Modela el ciclo de vida del estudiante: VerificandoConectividad, DashboardActivo, NivelBloqueado (candado visual si nota previa < 70%), NivelHabilitado, EvaluacionDeterminista y PersistenciaProgreso (con bifurcación entre GuardadoInmediatoFirestore y EncoladoLocalIndexedDB).

7.3 Algoritmos de Actividad (Diagramas 9.1 y 9.2)

- Actividad A (Generador de Preguntas): Selección pseudoaleatoria con semilla, permutación de marcas de caso y filtro estricto de distractores.
- Actividad B (Resiliencia Offline-First): Interceptación del Service Worker, almacenamiento no bloqueante en IndexedDB y vaciado atómico vía SyncManager.

CAPÍTULO VIII: MATRIZ DE AUDITORÍA Y GOBERNANZA DE IA (UPDS)

En cumplimiento de la normativa de integridad académica de la Universidad Privada Domingo Savio (Sede Santa Cruz), se documenta de forma transparente y verificable cómo se emplearon herramientas de Inteligencia Artificial en el diseño de este álbum de modelos, detallando las decisiones aceptadas, modificadas y rechazadas por el equipo humano:

Artefacto / Componente	Herramienta y Contenido Asistido por IA	Aceptado o Modificado por el Equipo	Rechazado Expresamente (Qué y Por Qué)
Diagrama 1: Arquitectura General (PWA + Firebase)	Claude / Gemini: Sugirió arquitectura cliente-servidor con integración de API externa comercial de OpenAI para generar exámenes.	Modificado: Se adoptó Astro SSG + Service Worker e IndexedDB para garantizar consumo RAM < 150MB y FCP < 1.8s.	Rechazado: Conexión externa a LLMs comerciales. Inviabile por coste de API (\$0 en SRS 3.3) y dependencia de internet continuo en áreas rurales.
Diagrama 2: Clases del Dominio Pedagógico	Asistente de IA: Propuso clases monolíticas acopladas con generación directa de texto en lenguaje natural.	Modificado: Se aplicó Inversión de Dependencias (IEvaluationEngine) y composición pura de Question dentro de Evaluation.	Rechazado: Instanciación directa de motores concretos; violaba el principio SOLID de inversión de dependencias para testing con Mocks.
Diagrama 3: Persistencia Multibase de Datos	Asistente de IA: Propuso usar una sola base de datos relacional MySQL tradicional sin soporte offline.	Modificado: Se diseñó persistencia dual: IndexedDB local con schemas indexados y Cloud Firestore NoSQL remota, vinculados por Data Mappers.	Rechazado: Forzar un único motor de base de datos; incapacitaba el funcionamiento offline-first en smartphones sin

señal de red.

**Diagrama 5 y 6:
Casos de Uso y
Moderación**

Asistente de IA:
Propuso umbral de
avance al 50% y
moderación por
votos comunitarios
abiertos estilo foro
(Reddit).

Modificado: Umbral del
70% obligatorio e
instauración del principio
de cuatro ojos (autor_id
!= validador_id) con
cuarentena.

**Rechazado:
Votación
comunitaria sin
filtro docente.
Habría
contaminado el
banco lingüístico
con interferencias
del castellano o
aymara.**

**Diagrama 4:
Modelo Entidad-
Relación (ERD)**

Asistente de IA:
Propuso guardar las
preguntas de
evaluación como un
texto JSON no
estructurado dentro
de la tabla de
usuarios.

Modificado: Se
normalizó la entidad
débil
DETALLE_PREGUNTA
con cardinalidad 1:10 y
claves foráneas tipadas.

**Rechazado:
Almacenamiento
desnormalizado sin
clave foránea;
impedía la auditoría
analítica de errores
morfológicos por
nivel.**

8.1 Evidencia de Mejora del Prompt

La interacción inicial con la IA partió de solicitudes abiertas de modelado UML genérico. El equipo detectó tempranamente que la IA tendía a proponer stacks de software convencionales pensados para entornos empresariales con conectividad de fibra óptica y presupuestos comerciales holgados (APIs de OpenAI, bases de datos SQL centralizadas, SPAs pesadas en React). Mediante refinamiento iterativo de prompts (*prompt engineering*), el equipo humano restringió el contexto incorporando las directrices de la stakeholder pedagógica (Lic. María Elena Quispe), la realidad técnica de las redes 2G/3G en Sucre y las restricciones formales de presupuesto (\$0) y memoria (< 150 MB RAM). Este proceso documenta el gobierno humano sobre los artefactos de software generados.

8.2 Análisis Crítico de Posibles Alucinaciones de la IA

En el dominio lingüístico de los idiomas originarios de Bolivia, los modelos de lenguaje masivos (LLMs) exhiben una tasa de error crítica superior al 35%, mezclando variantes dialectales (Collao con Cuzqueño o Ayacuchano) e inventando sufijos aglutinantes inexistentes. Para blindar el software contra estas alucinaciones, el equipo humano adoptó una decisión arquitectónica radical: descartar cualquier llamada a LLMs en tiempo de ejecución y sustituirlos por un motor determinista en TypeScript cuyas reglas y oraciones base provienen exclusivamente de docentes hablantes nativos certificados mediante el principio de los cuatro ojos.

CAPÍTULO IX: GUÍA DE EXPLICACIÓN PARA EVALUACIÓN ORAL (5 MINUTOS)

Para la defensa oral del trabajo grupal (30 puntos asignados al Criterio de Verificación #2), los dos integrantes del equipo han distribuido los 300 segundos de la exposición conforme al siguiente cronograma técnico riguroso:

Intervalo Temporal	Módulo / Vista Expuesta	Estudiante Responsable
0:00 - 0:45 (45 s)	Contexto Territorial & Arquitectura General PWA de 4 Capas	Emmanuel Ponce Quiroga
0:45 - 2:00 (75 s)	Diagramas de Clases (Dominio & Persistencia Multibase IndexedDB/Firestore)	Jhoel Alvaro Cruz Zurita
2:00 - 3:15 (75 s)	Motor IA Determinista en TS & Doble Moderación Docente (4 Ojos)	Emmanuel Ponce Quiroga
3:15 - 4:15 (60 s)	Vista Dinámica: Secuencia Temporal, Estados & Background Sync	Jhoel Alvaro Cruz Zurita
4:15 - 5:00 (45 s)	Matriz de Auditoría Ética de IA, Conclusiones y Ronda de Preguntas	Jhoel Alvaro Cruz & Emmanuel Ponce

9.1 Banco de Preguntas Críticas y Respuestas Técnicas de Defensa

1. ¿Por qué en el diagrama de arquitectura no colocaron un rombo para validar si hay internet?

Respuesta: Porque la arquitectura modela componentes estáticos y protocolos de comunicación (HTTPS TLS 1.3, HTTP/2, Cache API). En una PWA, el cliente siempre emite llamadas HTTP estándar que son interceptadas de forma transparente por el Service Worker; modelar un rombo

en la red es un error de abstracción conceptual que mezcla arquitectura con un diagrama de actividades procedimental.

2. ¿Cómo se modela la coexistencia de múltiples bases de datos en UML?

Respuesta: Aplicando el patrón Data Mapper y Repository. Se definen las entidades de negocio en el Dominio, mientras que los schemas de IndexedDB (ObjectStores con keyPaths e índices) y Firestore (Colecciones con marcas de tiempo) se modelan en capas de persistencia independientes. Mappers específicos traducen entre el dominio y cada base de datos, y un repositorio unificado coordina la escritura local y la sincronización remota.

3. ¿Por qué la relación entre Evaluación y Pregunta es Composición (*-->) y no Agregación (o-->)?

Respuesta: Porque las preguntas son sintetizadas dinámicamente con permutaciones específicas para ese intento evaluativo particular. No tienen existencia conceptual, clave primaria natural ni sentido de negocio fuera de la evaluación que las originó. Si la evaluación se descarta o se reinicia, sus 10 preguntas se destruyen en cascada en memoria.

CONCLUSIONES

1. Cumplimiento Estricto del Gate 3: El Álbum de Modelos UML de YAPU proporciona una descripción completa, modular y ejecutable del sistema bajo el estándar IEEE Std 1016-2009, sentando las bases definitivas para la fase de construcción.
2. Resiliencia y Pertinencia Social: La combinación de Astro SSG, Service Workers e IndexedDB asegura que los estudiantes de comunidades rurales de Chuquisaca puedan aprender quechua en sus propios dispositivos móviles sin depender de conectividad constante ni consumir planes de datos.
3. Soberanía Pedagógica: El motor determinista en TypeScript y el principio de doble moderación garantizan rigor lingüístico y coste cero, protegiendo la integridad dialectal del quechua Collao.

REFERENCIAS BIBLIOGRÁFICAS (APA 7MA EDICIÓN)

Cerrón-Palomino, R. (2003). Lingüística quechua (2.^a ed.). Centro de Estudios Regionales Andinos Bartolomé de Las Casas.

Constitución Política del Estado Plurinacional de Bolivia. (2009). Gaceta Oficial del Estado Plurinacional de Bolivia.

Elmasri, R., & Navathe, S. B. (2017). Fundamentals of database systems (7.^a ed.). Pearson.

IEEE Computer Society. (2009). IEEE Std 1016-2009: IEEE Standard for Information Technology — Systems Design — Software Design Descriptions. IEEE.
<https://doi.org/10.1109/IEEESTD.2009.5167255>

IEEE Computer Society. (2011). IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications. IEEE. <https://doi.org/10.1109/IEEESTD.1998.88286>

Ministerio de Educación del Estado Plurinacional de Bolivia. (2023). Currículo Base del Sistema Educativo Plurinacional. La Paz, Bolivia.

Pressman, R. S., & Maxim, B. R. (2020). Software engineering: A practitioner's approach (9.^a ed.). McGraw-Hill Education.

Sommerville, I. (2016). Software engineering (10.^a ed.). Pearson Education.

Wynne, M., & Hellesøy, A. (2017). The Cucumber book: Behaviour-driven development for testers and developers (2.^a ed.). Pragmatic Bookshelf.